Manual de programadores de Expedientes ETF

Autor: Expedientes ETF.

Este manual explica cómo mantener y ampliar Expedientes ETF. Está dirigido a quien vaya a cambiar el servidor Go, la interfaz React, las migraciones o los paquetes de idiomas, temas y plantillas. Describe la aplicación y sus pruebas; las reglas funcionales se consultan en los documentos de producto.

Capítulos

Capítulo	Qué encontrarás
1. Arquitectura y módulos	Dependencias entre capas, composición, contratos y transacciones
2. PostgreSQL y migraciones	Roles técnicos, RLS, historia, manifiestos y reserva de números
3. API HTTP, sesiones y CSRF	Registro de rutas, autorización, errores, versiones y recuperación
4. Interfaz React	Estructura, navegación, formularios, traducción y temas
5. Idiomas, catálogos, temas y Word	Directorios, publicación inmutable, marcadores y activación
6. Lectores de anexos y actas	DOCX, ODT, PDF, límites y revisión antes de guardar
7. Memoria, SPAF y auditoría	Lectura histórica, cálculo, cierres, Consulta y cadena de eventos
8. Pruebas y entrega	Go, PostgreSQL, navegador, F0.11, CI, demo, vídeos y Windows
9. Recetas de ampliación	Añadir un campo, pantalla, plantilla, idioma, catálogo o migración

El PDF del manual contiene estos capítulos, sus diagramas y capturas. La evidencia de edición recoge los comandos ejecutados y la procedencia de las capturas.

Antes de cambiar una capacidad

Lee PRD y PLAN_IMPLEMENTACION, además de los apartados pertinentes de TRD y ESQUEMA_BACKEND. El índice de producto permite encontrar sus rangos de líneas. Para una pantalla, consulta también FLUJO_APP y DISENO_UI_UX.

Busca después la capacidad en el código. El código determina qué está implementado; una ruta descrita como prevista en producto puede no estar registrada aún. Los puntos de entrada son cmd/etf/modulos.go, el registro HTTP y web/src/pantallas.ts. Sigue desde ellos el puerto de entrada, el caso de uso y sus puertos de salida.

Trabaja en la rama y carpeta asignadas. Conserva los cambios de otras tareas y limita la entrega a la capacidad encargada. Una decisión funcional pendiente se resuelve antes de convertirla en código o en una constante.

Preparar el PDF

Desde la raíz, con LibreOffice, Pillow y Graphviz instalados en el equipo:

```
TMPDIR="$PWD/bin/temporales" python3 -B docs/manuales/exportar.py --manual programadores
```

El exportador usa los capítulos de esta carpeta y escribe `docs/desarrollo/programadores.pdf`. La conversión es local. Graphviz dibuja los diagramas de flujo Mermaid de estos capítulos a partir de sus nodos y flechas. No descarga recursos. Sin `--manual`, el exportador conserva la selección de los tres manuales de usuario existentes.

Para guardar una edición de revisión fuera del repositorio:

```
TMPDIR="$PWD/bin/temporales" python3 -B docs/manuales/exportar.py \
  --manual programadores --salida bin/temporales/MANDEV-pdf
```

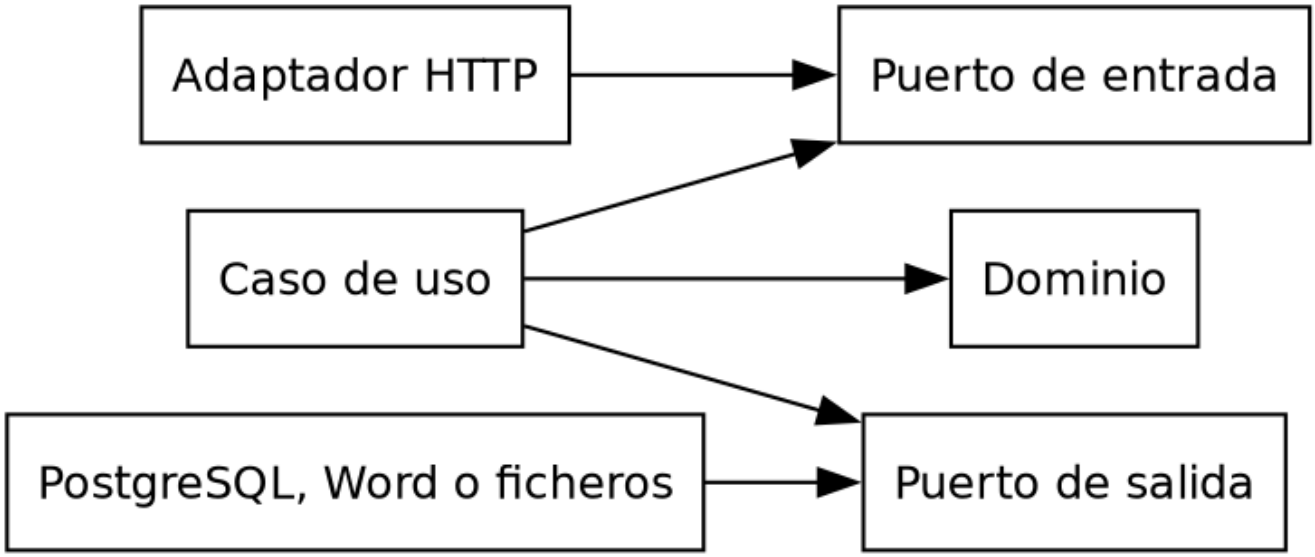
Las capturas se renuevan con la demo de este worktree, Playwright y Chrome del sistema. Pruebas y entrega explica el comando y las comprobaciones.

1. Arquitectura y módulos

Expedientes ETF es un monolito modular: el servidor reúne las capacidades en un proceso Go, con PostgreSQL como almacenamiento. Las capas separan las reglas de los sistemas que las ejecutan. Un cambio de lector, directorio de usuarios o motor Word se realiza en el adaptador que implementa su puerto.

Capas y dependencias

Carpeta	Responsabilidad	Dependencias permitidas
`internal/dominio/<modulo>/`	Entidades, valores, estados, cálculos y errores	Biblioteca estándar sin acceso a sistemas externos; fechas y datos recibidos como valores
`internal/aplicacion/<modulo>/puertos/entrada/`	Contratos que consume HTTP	Dominio
`internal/aplicacion/<modulo>/`	Casos de uso, autorización, transacción y auditoría	Dominio y puertos
`internal/puertos/salida/<modulo>/`	Interfaces hacia repositorios, lectores y otros sistemas	Dominio
`internal/adaptadores/http/`	Peticiones, límites de entrada y respuestas	Puertos de entrada; sin importar la implementación del caso de uso
`internal/adaptadores/postgres/`, `ficheros/`, `docx/`, `ldap/`, `clamav/`, `anexos/`	Implementaciones de los puertos externos	Dominio y puertos de salida, con las excepciones técnicas acotadas por la prueba de arquitectura
`cmd/etf/`	Crear dependencias e inyectarlas	Todas las capas necesarias para componer
`cmd/migrar/`	Ejecutar el plan de migraciones y terminar	Infraestructura de migraciones; separado del servidor



HTTP consume el puerto de entrada; el caso de uso depende del dominio y de los puertos de salida.

Las flechas indican dependencias del código. El caso de uso recibe implementaciones de sus puertos al arrancar. El dominio recibe la fecha como dato; el reloj de ejecución es un puerto de salida. Evita leer la hora, el entorno o un fichero desde una regla de dominio.

La prueba TestArquitectura inspecciona las importaciones de producción, incluidas variantes de plataforma. Consulta sus reglas antes de añadir una dependencia. Los dobles en memoria permiten probar casos de uso; una prueba con ellos no acredita persistencia ni RLS.

Fronteras de los módulos

Módulo	Datos y capacidades que posee
`plataforma`	Contexto técnico, transacciones y control de migraciones
`identidad`	Usuarios, perfiles, adscripción a equipo y sesiones
`catalogo`	Códigos, vigencias, equipos, municipios, centros, SMI, firmantes y destinatarios
`paquete`	Versiones y referencias activas de idiomas y temas; referencias a textos de catálogo
`expediente`	Derivación, aceptación, ficha, vigencias, hitos, salidas, cierre, traslado, seguimiento, carga y conservación
`evaluacion`	Instrumentos, momentos, sujetos y respuestas
`documento`	Plantillas, activaciones, emisiones y registro de salida
`actividad`	Comisiones e informes técnicos
`estadistica`	Tabla SPAF, Memoria, narrativas, revisiones y cifras
`auditoria`	Registro, cadena, historial y exportación de eventos

El directorio de migraciones incluye también `carga`, utilizado por la capacidad de importación de hojas. Esa carpeta técnica no añade un módulo al mapa de producto. Respeta su manifiesto y dependencias cuando trabajes en importación.

Cada módulo es dueño de sus tablas. Si otro necesita datos, consume su puerto o una proyección de lectura publicada por el propietario. El adaptador de estadística lee contratos históricos publicados por Expediente y Actividad; no sustituye esos contratos por consultas libres a las tablas internas. Las dependencias SQL se declaran por migración.

La comunicación entre capacidades ocurre dentro del mismo proceso. No hay colas ni bandejas de salida. El caso de uso que dirige la operación delimita una transacción común; las llamadas de colaboración pasan el mismo contexto.

Composición al arrancar

cmd/etf/modulos.go reúne la base ordinaria, la base técnica de cálculo, el antivirus, los paquetes de disco, la configuración y el registro técnico. Cada fichero de capacidad construye sus servicios y devuelve un módulo HTTP. `componerModulos` fija su orden explícitamente.

Los módulos HTTP aportan `Rutas() []Ruta`. componerMux añade plataforma, valida el registro y conecta las rutas de las pantallas con la página de la aplicación. No registres capacidades mediante `init()`: la lista de composición debe permitir ver qué entra en el servidor.

En la web, pantallas.ts compone las listas de cada capacidad. Sus rutas y perfiles se contrastan con las pantallas publicadas por el servidor. Añadir un componente sin registrar su pantalla deja la capacidad fuera de la navegación autorizada.

Transacción compartida

El puerto Transacciones recibe una identidad validada y una operación. Base.Ejecutar abre la transacción, fija el contexto RLS y pasa un contexto con el ejecutor a los repositorios y a auditoría. Confirma sólo cuando la operación termina sin error.

Las transacciones anidadas se rechazan. Un colaborador llamado dentro de una operación debe usar su variante de trabajo en la transacción existente. No abras otra conexión para registrar la auditoría de un cambio que aún puede revertirse. Tampoco devuelvas datos o empieces una descarga antes de confirmar.

Fuentes: TRD, arquitectura, mapa de datos y prueba de arquitectura.

2. PostgreSQL y migraciones

Las tablas viven en esquemas por módulo. La aplicación se conecta como `etf_app`; el despliegue prepara roles y aplica migraciones antes de arrancarla. PostgreSQL es la segunda barrera de autorización, además de los permisos que comprueban HTTP y los casos de uso.

Roles técnicos y perfiles

Rol PostgreSQL	Uso	Restricción que debes conservar
`etf_propietario`	Propietario sin inicio de sesión	No se usa como cuenta de aplicación
`etf_migrador`	Ejecutor puntual de migraciones	Sus credenciales quedan fuera del contenedor de la app
`etf_app`	Peticiones ordinarias	Sin propiedad ni `BYPASSRLS`;

Rol PostgreSQL	Uso	Restricción que debes conservar
		permisos concedidos objeto a objeto
`etf_calculo_memoria`	Cálculo interno de cifras para Consulta	Sin tablas personales, escritura, membresías ni `BYPASSRLS`
`etf_copia`	Copia completa	Sólo lectura; su `BYPASSRLS` se limita al servicio de copias

Los perfiles funcionales no son esos roles PostgreSQL. Coordinación, `profesional_etf`, Consulta y Administración viajan en el contexto de la transacción, derivados de la sesión. Administración mantiene usuarios y paquetes, pero no lee fichas. Consulta recibe una proyección de cifras sin nombres, composición ni narrativas. El profesional trabaja sobre su equipo; un perfil combinado conserva las restricciones de equipo aplicables a su sesión.

Contexto RLS

Al iniciar cada transacción, Base.Ejecutar fija:

```
SELECT set_config('app.usuario_id', $1, true),
       set_config('app.perfiles', $2, true),
       set_config('app.equipos', $3, true);
```

El tercer argumento `true` limita los valores a la transacción. La conexión vuelve al grupo sin identidad heredada de la petición anterior. El adaptador usa `DesdeContexto` para recuperar ese ejecutor; no debe consultar el grupo de conexiones por su cuenta dentro del caso de uso.

Las funciones de `plataforma` leen el contexto y las políticas deniegan cuando falta. Las tablas con datos personales o de equipos llevan `ENABLE ROW LEVEL SECURITY` y `FORCE ROW LEVEL SECURITY`. Prueba tanto lectura como escritura, con contexto vacío y con equipos distintos. Una lista vacía en la web no demuestra que una escritura ajena esté bloqueada.

Las funciones `SECURITY DEFINER` necesitan propietario sin login, `search_path` fijo, retirada de `EXECUTE` a `PUBLIC` y comprobación explícita del ámbito. Revisa el conjunto de políticas: varias políticas permisivas se combinan y pueden ampliar una lectura. Para Consulta, RLS filtra filas, pero no oculta columnas; su contrato requiere una proyección propia.

Conserva claves compuestas cuando un hijo pertenece a expediente, equipo y sujeto. Un UUID válido de otro expediente no autoriza a vincularlo como adulto o menor. Esas relaciones se prueban en SQL además de validarse en Go.

Historia, versiones e idempotencia

El expediente es la unidad de versión optimista para su ficha. Los cambios comparan la versión enviada y suben la versión al guardar. Un conflicto devuelve 409 y conserva el formulario para comparar. Algunas capacidades, como seguimiento y referencias activas, tienen además su propia versión.

Los datos históricos conservan vigencia y momento de conocimiento. Para un cálculo pasado, el lector debe seleccionar la revisión válida al corte y conocida en el instante requerido. Una corrección posterior no borra la revisión utilizada para una Memoria cerrada.

Las acciones que consumen número, confirman hitos o crean emisiones usan `Idempotency-Key`. Se conserva la respuesta asociada a usuario, operación y huella de petición durante el plazo del contrato. La misma clave y petición recuperan el recibo; cambiar el cuerpo con esa clave se rechaza. No implementes reintentos automáticos de escrituras en el adaptador PostgreSQL.

Plan de migraciones

Cada carpeta contiene SQL de cuatro cifras y un `manifiesto.json`. El lector del plan exige declarar todos los SQL y sus dependencias. El número crece dentro de cada módulo; los huecos reservados son válidos.

Una entrada real del manifiesto de Estadística es:

```
{
  "numero": "0007",
  "archivo": "0007_pendientes_lista_espera.sql",
  "depende_de": [
    { "modulo": "estadistica", "numero": "0006" },
    { "modulo": "expediente", "numero": "0027" }
  ]
}
```

El ejecutor ordena el grafo, rechaza ciclos y dependencias ausentes, toma un bloqueo asesor y ejecuta una transacción por migración. Registra módulo, número, nombre, huella y autor en `plataforma.migracion_aplicada`. La huella incluye los bytes SQL, un byte cero y el JSON canónico de dependencias ordenadas. Cambiar un SQL aplicado, su nombre o sus dependencias rompe la comprobación del historial.

`migracion.retroceso` significa que se intenta aplicar un número aún no registrado por debajo del último aplicado de su módulo. No se resuelve quitando registros del historial. Reserva otro número y conserva la secuencia de despliegue acordada.

Reserva y ensayo

Antes de crear el SQL, reserva el número en el registro de reservas de migraciones, fuera de Git, fuera de Git, indicando módulo, número, rama, descripción, fecha y estado `Reservada`. Usa el mecanismo de reserva indicado en el encargo. Nunca tomes una reserva ajena ni deduzcas que un hueco está libre porque no aparece un SQL en tu rama.

Después de integrar el `main` actual, comprueba que no haya aparecido un número superior al tuyo en el mismo módulo. Si ocurre, solicita al director un número nuevo y actualiza nombre, manifiesto y ensayo antes de entregar.

Una migración nueva comprueba el estado previo y añade permisos, RLS e integridad en el mismo fichero. No concede acceso a una tabla a medio preparar. Ensáyala en PostgreSQL 18 local, con instalación limpia y actualización desde el estado anterior, y con datos inventados. Requiere revisión independiente de SQL y permisos. Las correcciones se hacen con una migración posterior; la recuperación de un despliegue restaura su copia previa.

Los datos inventados de demo se siembran por comandos separados, no como datos personales dentro de una migración. Los códigos iniciales y reglas del catálogo sí pueden tener migraciones de datos.

Fuentes: roles y permisos, historia e idempotencia, migraciones y ejecutor.

3. API HTTP, sesiones y CSRF

El adaptador HTTP convierte una petición limitada y validada en una llamada al puerto de entrada. La identidad procede de la sesión del servidor. Los UUID y perfiles enviados por el navegador no sustituyen esa identidad.

Registro de rutas

Cada capacidad declara método, patrón, handler y regla de permiso en su módulo HTTP. El registro también declara las rutas de pantalla. El arranque rechaza duplicados, métodos vacíos, permisos incompletos y escrituras sin la protección exigida. Una ruta desconocida queda denegada.

Consulta internal/adaptadores/http para localizar la declaración actual. El esquema de API explica los contratos, pero incluye rutas de diseño: para comprobar disponibilidad, contrasta el módulo con la composición.

Capacidad	Ruta actual de referencia	Perfil
Sesión	`GET`, `POST`, `PUT` y `DELETE /api/sesion`	Entrada y sesión propia
Plantillas	`GET /api/admin/plantillas`	Administración
Publicación de plantilla	`POST /api/admin/plantillas/versiones`	Administración, con CSRF
Activación	`POST /api/admin/plantillas/{id}/activar`	Administración, con CSRF y versión previa
Bandeja de equipo	`GET /api/equipo/expedientes`	Profesional de su equipo y Coordinación
SPAF	`GET /api/estadistica/spaf?anio=&mes=&etf=`	Profesional de su equipo y Coordinación
Word SPAF	`POST /api/equipos/{etf}/tabla-spaf/{anio}/{mes}.docx`	Misma autorización; crea emisión
Memoria de Consulta	`GET /api/consulta/memoria/{anio}?equipo=`	Consulta; contrato numérico propio

No conviertas la generación de un Word en GET: crea una emisión y necesita protección de escritura. La confirmación de una firma o fecha real es otra acción; descargar un documento no confirma el hito.

Entrada y sesión

El adaptador LDAPS comprueba la cuenta y vincula el identificador estable del directorio. La aplicación decide perfiles y equipo. Una cuenta válida sin perfil queda pendiente de alta. La concesión inicial de Administración se realiza una vez y se audita; retirar después ese perfil no permite recuperarlo por reiniciar el servicio.

sesion.go define las cookies `__Host-etf-sesion` y `__Host-etf-csrf`. Son `Secure`, `HttpOnly`, con `Path=/` y `SameSite=Strict`. El token de sesión es opaco; el navegador no recibe una lista de permisos firmada que pueda usar como identidad independiente del servidor.

La respuesta de `GET /api/sesion` publica el estado, el testigo CSRF y las pantallas permitidas. SEG1 hace esta lectura pasiva: no renueva actividad. `PUT /api/sesion`, con CSRF y origen comprobados, continúa una sesión vigente sin desplazar el límite absoluto. Las respuestas autenticadas incluyen cabeceras con caducidad, máximo y hora del servidor para el aviso de dos minutos. web/src/sesion.tsx consulta esa respuesta y ajusta la navegación a `sin_sesion`, `pendiente_alta` o `activo`. No copies la contraseña ni el token de sesión a localStorage o sessionStorage.

Protección CSRF y origen

La configuración usa el origen HTTPS público exacto y una clave CSRF de 32 bytes. El servidor contrasta `Origin`, exige una única cabecera `X-CSRF-Token` y comprueba el testigo mediante HMAC y comparación

constante. Cuando existe `Sec-Fetch-Site`, debe indicar el mismo origen. Una petición de escritura sin sesión no obtiene permiso por llevar un token inventado.

El origen no se deriva de `Host` ni de `Forwarded` del cliente. Para la IP, sólo se admite `X-Forwarded-For` desde los proxies configurados; una cadena mal formada invalida la petición. Conserva esas comprobaciones al añadir un proxy o un endpoint.

La demo local adapta su origen exacto a ese contrato HTTPS mediante su proxy propio. Ese comportamiento se limita al entorno de demostración; no es una instrucción para desactivar cookies seguras en un despliegue.

Petición, errores y recibos

Limita el cuerpo antes de decodificarlo. Valida formato, campos y tipo real de archivo. Pasa el contexto de cancelación al caso de uso. El handler no ejecuta SQL ni decide por sí solo si una persona puede modificar un expediente.

Los errores llevan claves y, cuando el contrato lo necesita, parámetros y campos. La interfaz los resuelve en `idiomas/<codigo>/textos/`. Reutiliza el formato del handler de la capacidad: existen respuestas con clave simple y otras con `error.clave`. La API de Plantillas admite ambas. No añadas frases castellanas a una respuesta Go.

Estado HTTP	Significado en la aplicación
400	Cuerpo o parámetros mal formados
401	Falta una sesión válida
403	La operación no está autorizada
404	Recurso inexistente o fuera del ámbito, según el contrato
409	Conflicto de versión, estado o reproducción
413	Cuerpo o archivo por encima del límite
415	Formato no admitido
422	Incumplimiento de una regla o reutilización incompatible de idempotencia
503	Servicio necesario no disponible

Las acciones devuelven un recibo con sus identificadores, versión y resultado. Ante una respuesta perdida, conserva la misma clave de idempotencia y el mismo cuerpo para recuperar la acción. Una petición modificada necesita una clave nueva. Ante 409, compara la versión actual y permite revisar el formulario; no repitas el guardado a ciegas.

Para una lectura protegida, reúne el resultado en memoria, registra el evento y confirma la transacción antes de escribir la respuesta. Si falla la auditoría, devuelve el error sin datos. La misma regla se aplica a descargas y exportaciones.

Fuentes: sesión y autorización del TRD, API y F0.11.

4. Interfaz React

La interfaz usa React, TypeScript y Vite, con controles accesibles de React Aria. Las capacidades se organizan por carpeta y publican su pantalla en un registro común. Los textos se leen de la API de idiomas y el aspecto procede de los tokens del tema.

Archivos que conectan la aplicación

Archivo o carpeta	Función
main.tsx	Montaje, carga de idiomas y temas, proveedor de localización y estado general
sesion.tsx	Entrada, sesión, pendiente de alta y elección de ruta autorizada
pantallas.ts	Lista explícita de pantallas de las capacidades
registro-pantallas.ts	Contrato, validación, coincidencia de rutas y permisos de navegación
armazon.tsx	Cabecera, menú, persona conectada y preferencias
cambios.tsx	Protección de formularios con cambios sin guardar
plantillas/	Pantalla, API, pruebas y estructura visual de Plantillas Word
equipo/	Bandeja y ficha integrada del equipo

Una `Pantalla` declara `ruta`, `perfiles`, `clave` y `componente`. Puede declarar `menuDe` para una ruta subordinada y `conservarAlNavegar` para conservar el formulario. `PropsPantalla` entrega traducción, idioma, CSRF y funciones de actualización y navegación. Reutiliza ese contrato antes de crear un estado global paralelo.

El menú exige coincidencia entre el registro web y las pantallas autorizadas por la sesión del servidor. Ocultar un enlace mejora el recorrido, pero las peticiones siguen necesitando autorización HTTP, aplicación y RLS.

Datos y formularios

Cada capacidad mantiene su cliente API y tipos cerca de la pantalla. plantillas/api.ts muestra cómo enviar credenciales del mismo origen, interpretar la clave de error y mandar CSRF. Evita convertir un fallo HTTP en una lista vacía: carga, vacío y error deben tener estados distintos.

En un formulario, separa el borrador de los datos confirmados. Muestra un recibo al guardar y recupera la versión. Si llega un conflicto, conserva lo escrito y ofrece comparar. La protección de cambios sin guardar debe cubrir navegación, pestañas y salida de sesión cuando afecten al formulario.

Las fechas de firma, registro o cierre se confirman por sus acciones propias. Un botón de descarga no debe rellenar esas fechas. Para guardar o generar, conserva la clave de idempotencia mientras la respuesta sea incierta.

Usa controles con etiqueta, ayuda y error asociado. Devuelve el foco al resultado o al campo que requiere corrección. Comprueba el recorrido con teclado y a 390 píxeles: la persona debe poder leer un error, alcanzar la acción principal y comparar sin desplazamiento horizontal de toda la página.

Traducción y tema

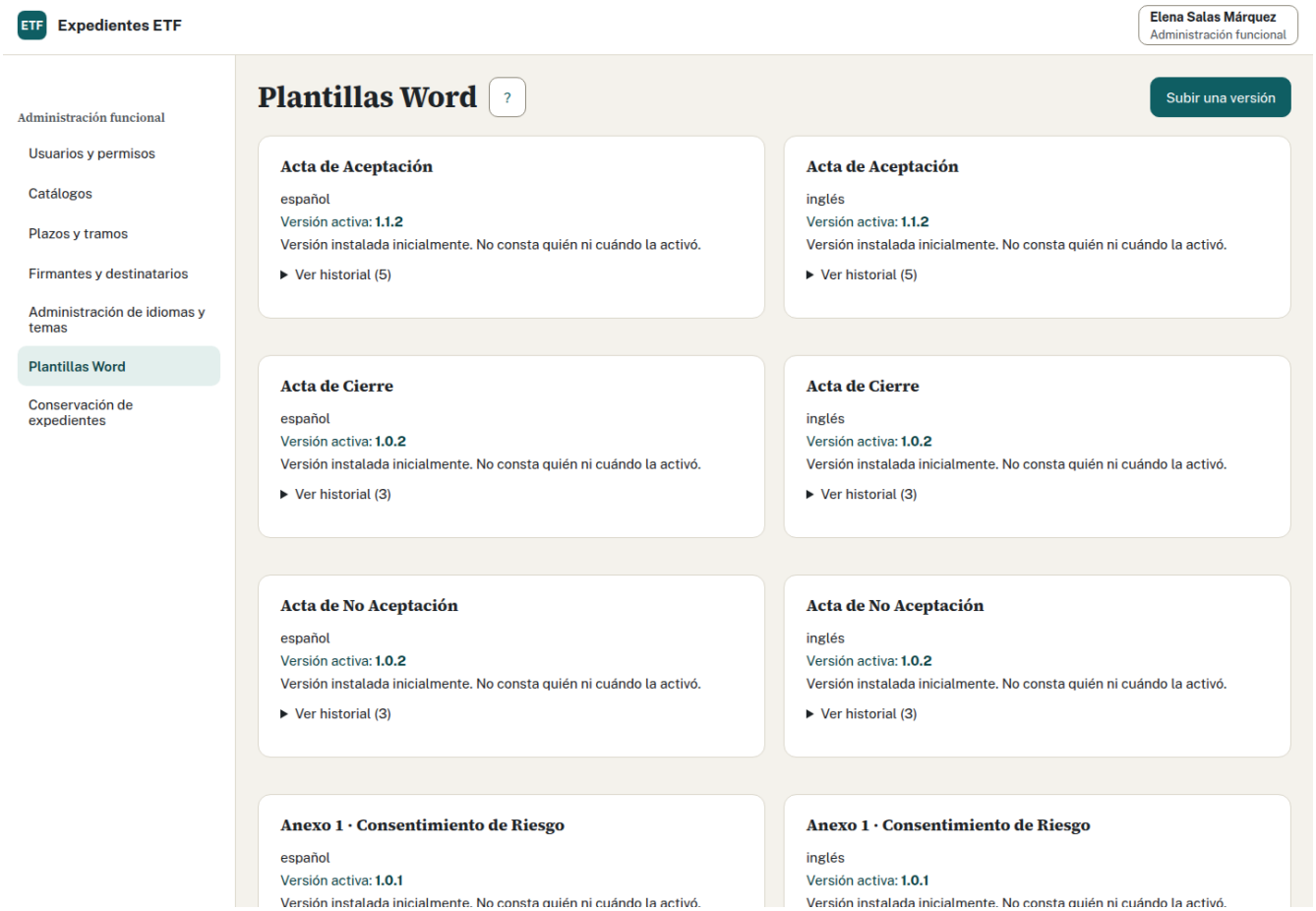
`t(clave)` busca el texto cargado. Las etiquetas, botones, títulos, `aria-label`, ayudas y mensajes deben usar claves presentes en los dos idiomas mantenidos. No escribas texto visible en JSX ni en errores JavaScript. Los nombres y narrativas introducidos por una persona son datos del expediente, sujetos a sus permisos.

`main.tsx` obtiene idiomas y textos desde `/api/idiomas` y cambia `document.documentElement.lang` y el título con esos datos. El proveedor de React Aria recibe la localización. Usa `Intl` para fechas y números según el idioma seleccionado.

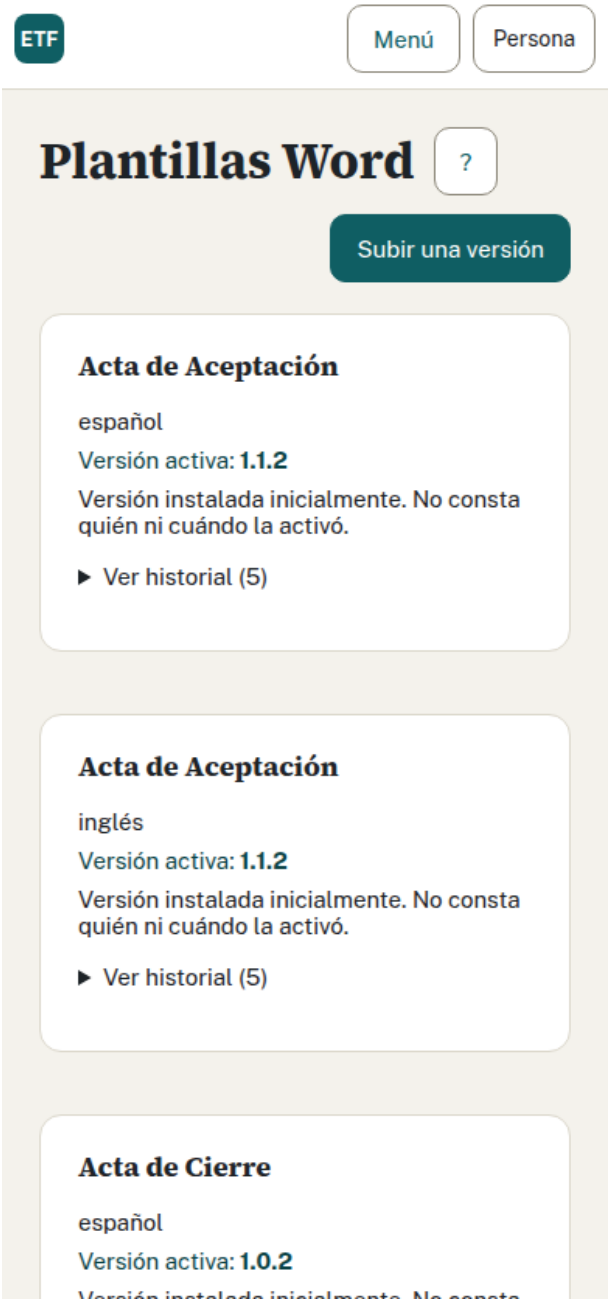
Los estilos de estructura pueden distribuir columnas y controles, pero los valores de tema se expresan con variables semánticas. Colores, fuentes, espaciado, radios y alturas salen de `temas/<nombre>/`. La hoja del tema se sirve desde `/api/temas/{nombre}/tokens`. No copies valores de color del diseño a un CSS de componente.

Una misma capacidad en dos tamaños

Las capturas muestran la pantalla de Plantillas Word de una demo local. El listado y el historial proceden de los mismos contratos en ambos tamaños. El cambio de disposición mantiene los controles de publicación separados de la activación.



Plantillas Word a 1440 píxeles: tipos, versiones activas y acciones de administración en la demo local.



Plantillas Word a 390 píxeles: controles apilados y listado accesible sin desbordar la página.

Comprobaciones de interfaz

Al cambiar una capacidad, ejecuta sus pruebas y la compilación TypeScript. Añade una prueba de comportamiento cuando haya nueva lógica de guardado, conflicto o navegación. El recorrido real debe usar Chrome del sistema contra la demo, con datos inventados. Para accesibilidad, consulta el alcance F5.03; las comprobaciones automáticas no sustituyen el uso con teclado y lector de pantalla.

Fuentes: TRD, textos, diseño de interfaz, registro web y estructura de pruebas.

5. Idiomas, catálogos, temas y Word

Los textos y el aspecto son paquetes de datos. PostgreSQL conserva códigos, metadatos y referencias activas; los archivos contienen los textos, los recursos y las plantillas. Publicar una versión y activarla son operaciones distintas cuando el contrato de la capacidad las separa.

Directorio de un idioma

```
idiomas/<codigo>/
  idioma.json
  textos/
  catalogos/
  plantillas/
```

`idioma.json` declara código, nombre y versión. En el castellano de distribución, la versión del paquete es `0.1.0`. Ese número no es la versión de cada plantilla Word.

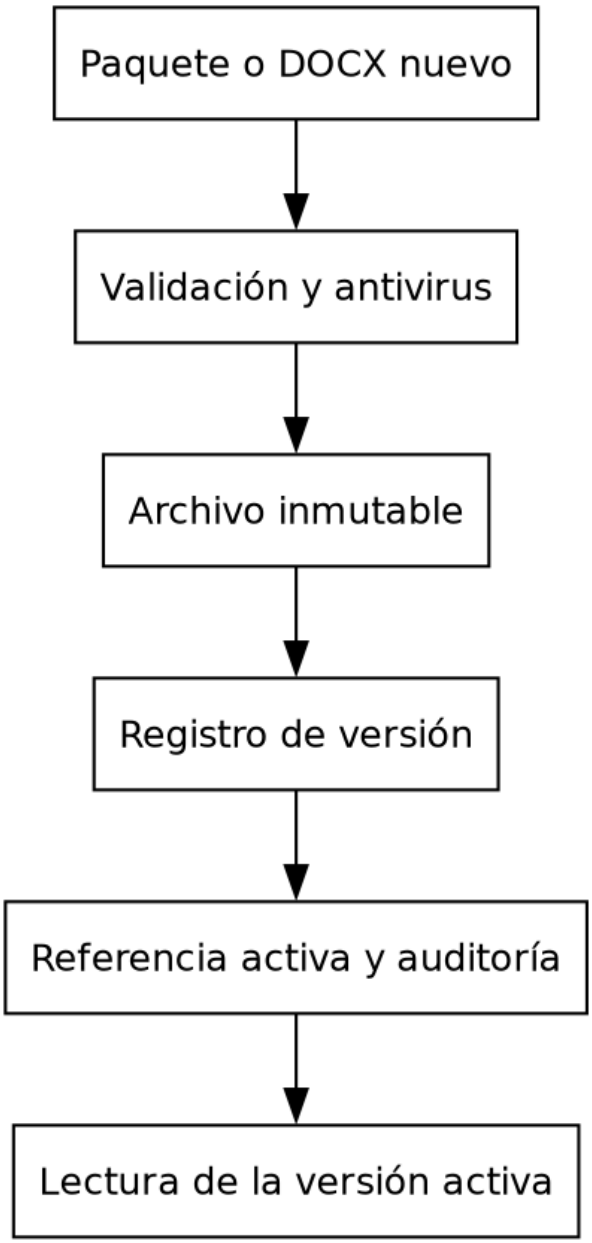
`textos/` contiene JSON de pantalla, ayudas, errores y avisos, separados por capacidad. El código sólo conoce claves. Toda clave nueva necesita sus textos en los idiomas mantenidos y debe superar `make textos`. Las claves deben ser únicas dentro de un idioma.

`catalogos/` contiene etiquetas de opciones. La base guarda el código de municipio, motivo, equipo o lista y la referencia a sus versiones; no guarda traducciones. La aplicación resuelve la etiqueta del código en el idioma seleccionado, con respaldo del predeterminado cuando corresponde.

`plantillas/` contiene DOCX por tipo e idioma. Las emisiones conservan la versión y huella utilizadas. Los idiomas y las plantillas históricas referenciadas entran en las copias y deben sobrevivir a una actualización.

Los textos libres de la ficha y las narrativas de Memoria sí son datos de negocio en PostgreSQL. La prohibición de textos en el código no desplaza esos datos al paquete de idioma.

Publicación inmutable



La validación precede al archivo inmutable; la referencia activa se confirma con su auditoría.

La escritura crea una ruta nueva, sin sobrescribir un archivo activo. Después se registra su metadato y, al activar, se compara la versión previa de la referencia. La activación y su auditoría se confirman juntas. Si falla, sigue activa la versión anterior. Un archivo huérfano no se sirve y se limpia según el resultado confirmado de la operación.

El acceso a una versión publicada no debe depender de que el paquete actual de idioma mantenga el mismo archivo. Una emisión antigua necesita sus bytes y contratos originales para reproducirse.

Catálogos con vigencia

Los catálogos mantienen borradores y versiones publicadas con vigencia. Las reglas de cálculo se reciben como datos: plazos, tramos, equivalencias, SMI y motivos no se convierten en constantes Go. Las etiquetas se publican en los directorios de idioma mediante las capacidades de paquetes.

Una opción nueva exige dos piezas: código y regla en Catálogo; etiqueta en cada idioma. Para una lista con relación estadística, define también su equivalencia y tratamiento de desconocidos. No cambies el significado de un código histórico para reutilizarlo.

El cálculo de un año elige las reglas vigentes al corte que establece el contrato. Un catálogo publicado hoy no debe alterar una revisión cerrada de Memoria. Consulta las versiones de catálogo y sus ensayos PostgreSQL.

Temas

Cada tema vive en `temas/<nombre>/`, con `tema.json`, tokens semánticos y los recursos admitidos. El adaptador genera variables CSS después de validar tipos, rangos y contraste. El paquete no trae CSS libre, JavaScript ni SVG. Los recursos admitidos son PNG y fuentes WOFF2, con límites y controles de archivo.

Administración instala una versión, activa su referencia y puede retirarla mediante las capacidades existentes. La aplicación conserva un tema de reserva. Al añadir un token, revisa todos los temas mantenidos, su validador y el uso en la web; `make temas` comprueba coherencia y contraste.

La capacidad de instalación en escritorio utiliza manifiesto e iconos derivados de textos y tokens. scripts/pwa1/iconos.mjs regenera los iconos. El service worker no conserva respuestas de expedientes y ofrece un aviso traducido cuando falla la conexión.

Plantillas y marcadores

El motor internal/adaptadores/docx valida y rellena DOCX. Comprueba el ZIP y XML, rechaza macros y contenido activo y limita recursos externos conforme a su lista cerrada. Los valores se escapan al escribir XML. Los marcadores divididos por Word entre fragmentos de texto también forman parte de la validación.

Para una versión nueva del mismo tipo, conserva el contrato esperado de campos y bloques. La publicación informa de marcadores que faltan o sobran. Un tipo nuevo necesita contrato, generador, permisos y pruebas además del archivo; subir un DOCX no crea una capacidad documental.

La ruta de publicación actual es:

```
idiomas/<codigo>/plantillas/<tipo>/.versiones/<UUID>/v<version>.docx
```

`POST /api/admin/plantillas/versiones` recibe el DOCX como cuerpo, con su Content-Type y parámetros `tipo`, `idioma`, `version` y `version_previa`. ClamAV debe confirmar que está limpio. Se guarda sin activar. `POST /api/admin/plantillas/{id}/activar` recibe `idioma` y `version_previa`, valida la versión elegida y cambia la referencia tras la confirmación de la persona.

El historial conserva autor, fecha y activaciones, incluidas restauraciones de una versión previa. La restauración cambia la referencia; no reescribe el archivo anterior. La migración inicial o una actualización de distribución debe preservar las publicaciones y decisiones administrativas existentes.

La emisión conserva metadatos y huella, pero no guarda el Word generado. Una recuperación idempotente regenera con datos y plantillas fijados y comprueba que los bytes coinciden; si no puede reproducirlos, devuelve el recibo y el conflicto correspondiente. Generar de nuevo con otros datos crea otra emisión.

Fuentes: TRD, plantillas, publicación de archivos, gestión PostgreSQL y revisión por plantilla.

6. Lectores de anexos y actas

Los lectores proponen datos para revisar. La subida de un anexo no crea un expediente: el caso de uso valida el archivo, pasa el antivirus, lee en memoria y devuelve campos, dudas y propuestas de catálogo. La persona confirma los datos en una acción posterior. El archivo recibido no se conserva.

Entrada y revisión

El lector compuesto internal/adaptadores/anexos/lectura implementa el puerto de Expediente y limita la lectura a una operación simultánea. Selecciona DOCX, ODT o PDF por el formato validado. No confíes sólo en el nombre ni en el Content-Type proporcionado por el navegador.

El resultado contiene reconocimiento de plantilla, tipo, versión, avisos y campos. Cada campo lleva código, valor, página cuando se conoce, marca de duda y motivos. Esos motivos son claves que la interfaz traduce. Un campo ausente permanece pendiente de completar; no lo rellenes con una suposición.

Tras la revisión, el alta comprueba otra vez el ámbito, los códigos y las reglas. Descartar elimina la revisión temporal. La confirmación guarda datos estructurados, su procedencia y la auditoría; no guarda el anexo.

DOCX y ODT

El lector DOCX reconoce los Anexos 3 y 4 mediante perfiles de formato y extrae contenido del ZIP y XML limitado. Conserva las dudas de campos que no puede identificar con suficiente precisión. Los corpus de prueba están junto al adaptador.

El lector ODT trabaja con los formatos de LibreOffice admitidos y sus perfiles. La migración `expediente/0026` incorpora el formato ODT al contrato persistente sin cambiar permisos. No conviertas el archivo con una aplicación de escritorio durante una petición del servidor.

Al ampliar un perfil, añade una muestra inventada y su resultado esperado: tipo reconocido, campos recuperados, dudas y vacíos. Incluye formato no reconocido, ZIP corrupto y límites de expansión. Un documento que se abre en Word puede seguir incumpliendo el contrato del lector.

PDF

El adaptador PDF usa un trabajador efímero con entrada y salida por tuberías. Tiene concurrencia y recursos acotados; no escribe nombres ni contenido en archivos temporales. El lector limita la entrada a 10 MiB y el análisis a 20 páginas, además de objetos, lecturas y glifos.

El trabajador fija límites de memoria y CPU. Las pruebas Linux comprueban esos límites. Conserva el mecanismo al añadir extracción; pasar el PDF al proceso principal para acelerar un caso quitaría la protección frente a archivos costosos.

Se admiten PDF con texto y los formatos reconocidos de LibreOffice, incluidos los contrastados con firma digital local. La lectura de texto no verifica la validez jurídica de una firma. Un PDF escaneado devuelve el aviso para completar datos a mano. No hay reconocimiento óptico de caracteres en este recorrido.

La evidencia RESP4 muestra valores recuperados y campos que requieren revisión. `probar_subida.py` del adaptador permite ensayar HTTP con ClamAV de Compose y recursos desechables.

Actas históricas

La lectura de actas es una capacidad separada para preparar carga histórica. LeerActa dirige DOCX y ODT a sus lectores correspondientes y PDF al trabajador limitado. El extractor de actas aplica su perfil y devuelve campos de aceptación o cierre. El lector de actas ODT recupera líneas del XML de LibreOffice y utiliza ese mismo contrato; su confirmación se ensaya en el recorrido de carga.

Su resultado prepara un alta histórica; no ejecuta automáticamente aceptación, cierre o firma ni consume numeración nueva. Confirma números y fechas originales con los mismos servicios de carga. El contrato RESP2 explica las rutas y los límites. La web y el recorrido están en carga y carga-actas.mjs.

Antivirus y errores

Un resultado positivo del antivirus bloquea el archivo. La indisponibilidad tiene un contrato distinto según la operación: la lectura de anexos contempla el aviso explícito permitido; la publicación de plantillas y paquetes y la importación de hojas requieren confirmación de limpieza. No extiendas la excepción de anexos a otra entrada.

Prueba un archivo limpio, EICAR, formato disfrazado, XML o ZIP excesivo, cancelación, lector ocupado y trabajador agotado. Comprueba además que un error no guarda el archivo ni crea un expediente parcial. Las

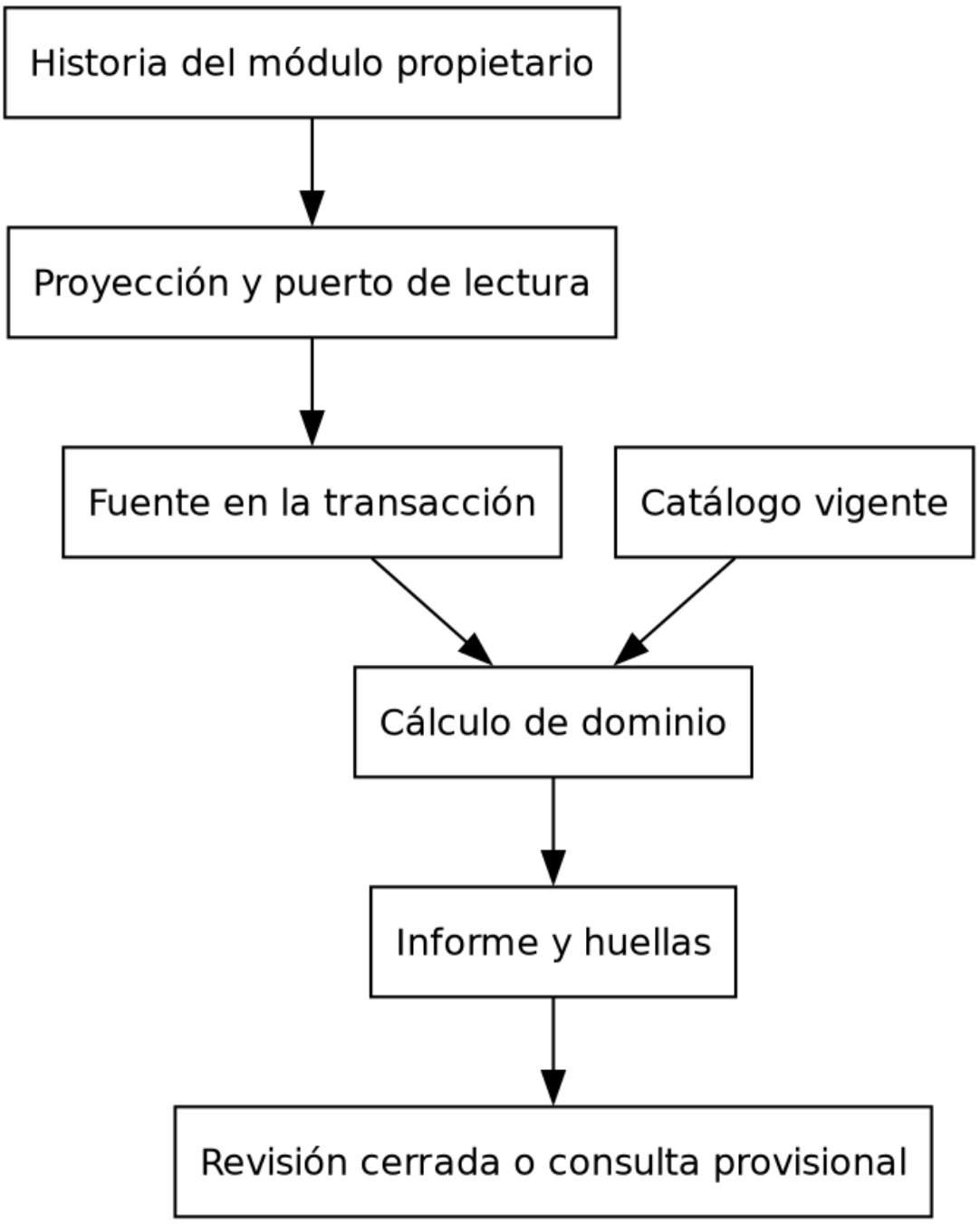
evidencias y descargas de prueba deben quedarse fuera de Git salvo los corpus inventados y las capturas encargadas.

Fuentes: entradas de archivos, lectores y subida HTTP PDF.

7. Memoria, SPAF y auditoría

Memoria y SPAF calculan a partir de datos confirmados y sus vigencias. Cada cálculo tiene un contrato de población, fecha, unidad y tratamiento de desconocidos. Añadir un campo a la ficha no lo incorpora por sí solo a una cifra: debe atravesar la proyección histórica y la entrada del cálculo.

De la ficha al cálculo



Las proyecciones históricas y el catálogo vigente alimentan el cálculo; el cierre conserva una revisión.

FuenteMemoria reúne lectores de Expediente, reglas, Actividad, narrativas históricas y equipos. Usa la identidad autorizada y el contexto de una transacción compartida. El dominio internal/dominio/estadistica recibe esos datos como valores y no consulta PostgreSQL.

El contrato por apartado fija población, denominador, desconocidos, fecha de datos y regla. contrato.go conserva las claves de apartados y unidades. Comprueba el contrato antes de tocar una fórmula; un total aparentemente correcto puede mezclar poblaciones o cortes distintos.

Los recuentos y porcentajes conservan por separado si el denominador es conocido. `CalcularPorcentaje` devuelve `estadistica.denominador_pendiente` cuando falta la base y `estadistica.divisor_cero` cuando es cero. Con base positiva, redondea `numerador / denominador × 10000` y devuelve centésimas de porcentaje. Rechaza negativos y desbordamiento. El motor no sustituye una base pendiente por el total visible de otra fila; conserva las pruebas de esos estados cuando añadas una base.

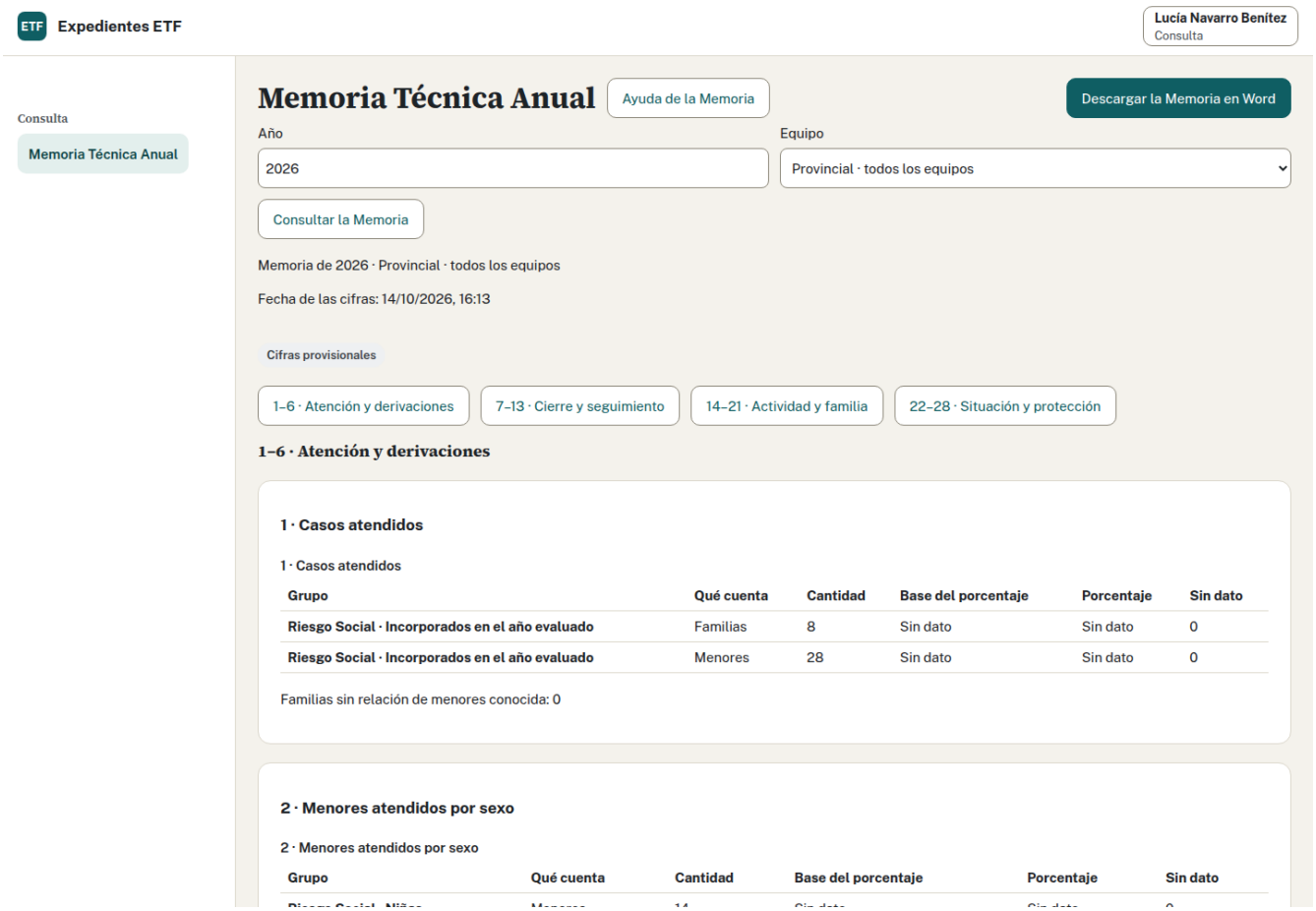
Para cambios retroactivos, distingue la vigencia del dato del momento en que se conoció. El lector histórico conserva episodios de cierre y reapertura y las revisiones de familia, adultos, menores e intervención. El traslado tiene origen y destino independientes. La Memoria cerrada debe poder reproducir el corte que utilizó.

Cálculo y cierre de Memoria

La consulta abierta calcula al momento y se presenta como provisional. El cierre fija una revisión con cifras, reglas, huellas y narrativas. Reabrir conserva las revisiones anteriores; una corrección requiere un cierre posterior para convertirse en otra foto definitiva.

El cierre de equipo corresponde a su profesional autorizado; Coordinación cierra o reabre dentro de su ámbito, incluida la provincial. El cierre provincial comprueba las revisiones de equipos exigidas por el contrato. Mantén la comprobación de discrepancias entre una foto cerrada y los datos corregidos después.

Las narrativas A, D, E y F son datos de negocio. Se guardan con versión y permisos. El Word completo combina cifras y narrativas con su plantilla; generar el Word crea una emisión y no cierra la Memoria.



Memoria de Consulta a 1440 píxeles: año, ámbito y cifras provisionales, sin narrativas ni composición de expedientes.



Memoria de Consulta a 390 píxeles: controles de año y ámbito, descarga y navegación por bloques adaptados al ancho móvil.

Consulta recibe sólo cifras

F4.07 tiene un tipo de salida propio para Consulta. Una Memoria cerrada devuelve su foto numérica; una abierta usa el ejecutor técnico con `etf_calculo_memoria`. La credencial se configura con `ETF_CALCULO_MEMORIA_CONTRASENA` y el adaptador valida que el rol no tenga privilegios o membresías indebidos.

El resultado de Consulta excluye nombres, UUID de expedientes, composición y textos libres. Su Word es exclusivamente numérico. No reutilices el JSON del informe completo quitando campos en React: el límite se aplica en servidor y en los permisos de PostgreSQL.

Coordinación y el equipo pueden consultar la composición autorizada de una cifra por números de expediente mediante el contrato de navegación. Eso no concede la misma capacidad a Consulta. Revisa la evidencia F4.07 y las pruebas negativas antes de ampliar cualquiera de estas salidas.

Tabla Mensual SPAF

SPAF utiliza un modelo propio con datos personales autorizados y episodios confirmados. Sus cuatro recuentos son altas, bajas, casos y menores, con fechas al corte y reglas de catálogo. Un caso aceptado y cerrado en el mes debe probarse expresamente.

Recuento	Cómo lo obtiene `CalcularSPAF`
Casos	Un expediente cuenta una vez si tiene un episodio aceptado y abierto antes o durante el mes, cuyo cierre no sea anterior al primer día del mes
Altas	La aceptación inicial cae dentro del mes, incluidos sus dos extremos
Bajas	Existe al menos un cierre de episodio dentro del mes; varios cierres del mismo expediente no multiplican su baja
Menores	Suma el número conocido de menores de la ficha histórica al fin del mes; los casos sin ese dato se conservan en un recuento y aviso separados

El mes debe haber terminado antes de la fecha de cálculo. Las reglas y la ficha se seleccionan al último día del mes y según el instante de conocimiento solicitado. Por eso el total de casos no equivale sólo a los expedientes que siguen abiertos al terminar el mes.

La vista previa y el Word fijan las versiones necesarias. Descargar exige POST, CSRF e idempotencia y confirma emisión y auditoría. El traslado no borra el movimiento de origen. La plantilla actual muestra el municipio según el contrato; evita introducir datos ajenos a sus marcadores.

Para añadir una dimensión, modifica el puerto de lectura y su productor, el dato de entrada del dominio, las reglas y la salida, y sólo después la plantilla. Comprueba que los menores y la fase recuperados sean los vigentes en el mes elegido. El recorrido F3.06/F3.07 conserva los fallos detectados; su ejecución puede exigir corregir capacidades que la CI general no cubre aún.

Auditoría encadenada

La tabla `auditoria.evento` se escribe mediante `auditoria.registrar`. La función toma un bloqueo asesor de transacción, obtiene la huella anterior, calcula SHA-256 de esa huella y la forma canónica del evento e inserta el siguiente. El dominio de cadena permite comprobar la misma representación.

`etf_app` no inserta, cambia ni elimina eventos directamente. La exportación y la verificación tienen funciones y permisos propios. El historial del expediente es una proyección limitada, con ámbito comprobado y máximo de acciones recientes.

El cambio y su evento comparten transacción. Para lecturas, se confirma el evento antes de entregar el resultado. Los intentos fallidos se registran después de revertir la operación, en otra transacción. Si también falla ese registro, la respuesta no entrega datos y el registro técnico conserva sólo el diagnóstico mínimo.

El detalle lleva identificadores, códigos de acción, nombres de campos y claves de error. No añadas nombres, direcciones, contraseñas ni narrativas. Los identificadores y sus vínculos siguen requiriendo protección; que una auditoría omita nombres no permite publicarla.

Pruebas de cálculo

Calcula el resultado esperado aparte, con familias inventadas y pequeñas. Prueba divisor cero, datos desconocidos, mayoría de edad durante el año, corte al cierre, reapertura, traslado y corrección conocida después del cierre. Un dato desconocido no es cero ni una lista vacía.

Los casos de estadistica/pruebas conservan resultados independientes. La lectura PostgreSQL necesita ensayos de las mismas vigencias y aislamiento entre equipos. F4.09 añade el recorrido de los 28 apartados, Word, cierres, corrección y reapertura.

Fuentes: consistencia del TRD, Estadística y Auditoría y auditoría en datos.

8. Pruebas y entrega

Las pruebas de una capacidad deben mostrar el comportamiento que se entrega. Un test Go con repositorio en memoria comprueba reglas y coordinación, pero no acredita RLS, persistencia ni una pantalla utilizable. Selecciona esas pruebas según el cambio y termina con la puerta completa `make ci`.

Preparación local

Las versiones del servidor están fijadas en go.mod y go.sum. La interfaz usa package.json y package-lock.json. Conserva las sumas y no instales un navegador alternativo para los recorridos.

La CI necesita Git, Bash, Python, Go, Node con npm, Docker, Gitleaks y Chrome del sistema. Las versiones y requisitos completos están en scripts/ci/README.md. Usa la caché Go predeterminada del equipo; no sitúes `GOMODCACHE` en memoria compartida. En este entorno los temporales se crean en `/dev/shm/etf/tmp`.

Pruebas por cambio

Cambio	Comprobaciones necesarias
Regla o cálculo Go	Pruebas del paquete, límites y resultados esperados independientes
Caso de uso	Permisos, transacción, reversión, auditoría, versión e idempotencia
PostgreSQL	Instalación y actualización, restricciones, RLS, privilegios y concurrencia reales
Pantalla	Tipos, pruebas de comportamiento, navegador, teclado y estados de carga, vacío, error y conflicto
Idioma o tema	Paridad y existencia de claves; tokens y contraste; lectura de la versión activada
Plantilla	Marcadores, seguridad DOCX, reproducción, activación y revisión del Word
Permiso o endpoint	Anónimo, perfil equivocado, equipo ajeno, método prohibido, CSRF y ruta desconocida
Manual o exportador	Enlaces, archivos, PDF legible y conservación de los manuales anteriores

Desde la raíz:

```
go test ./internal/dominio/estadistica/...
go test ./internal/aplicacion/estadistica/...
go vet ./...
go test ./...
make textos
make temas
python3 -B scripts/ci/enlaces.py
```

Para la interfaz, usa los comandos de `web/package.json` y el filtro apropiado de sus pruebas:

```
cd web
npm ci --ignore-scripts --no-audit --no-fund
npm test
npm run build
```

Los ensayos PostgreSQL están junto a las migraciones. Consulta su alcance antes de ejecutar uno: pueden levantar contenedores desechables y necesitan Docker. Entre las puertas actuales están:

```
python3 -B migraciones/catalogo/ensayar.py
python3 -B migraciones/expediente/ensayar.py
python3 -B migraciones/documento/ensayar.py
python3 -B migraciones/estadistica/ensayar.py
```

Una migración sensible requiere además ensayo en el clon local con datos inventados y revisión independiente. Las revisiones de seguridad se ejecutan en local; no envíes código ni datos a servicios de análisis externos.

Recorridos y permisos F0.11

```
python3 -B scripts/recorridos/entrada_permisos.py
make recorridos
```

F0.11 arranca la demo de este worktree, obtiene el registro HTTP real y recorre entrada y alta con Chrome del sistema. Contrasta todos los métodos declarados con cada perfil y comprueba RLS en PostgreSQL. La matriz incluye anónimo, perfil incorrecto, equipo ajeno, métodos no admitidos, CSRF y rutas desconocidas. Conserva evidencia fuera de Git. El registro de rutas no sustituye las pruebas negativas independientes.

`make recorridos` ejecuta entrada y permisos, ficha integrada, salidas y SPAF, Memoria integrada y accesibilidad. F0.11 también forma parte de `make ci`; los demás recorridos conservan sus propios criterios y pueden detectar fallos pendientes fuera de esa puerta. No presentes un recorrido fallido u omitido como aprobado. Consulta F0.11, F2.07, F3.06/F3.07, F4.09 y F5.03.

CI local y GitHub Actions

```
make ci
```

scripts/ci/comprobar.sh comprueba catálogos, índice de producto, enlaces, guiones, secretos, TypeScript, pruebas web, textos, temas, `go vet`, pruebas Go y compilación. Después ejecuta los ensayos PostgreSQL, la incorporación de idiomas y temas a la demo y los recorridos de entrada, permisos y carga. La comprobación de arranque exige TCP autenticado y evita aceptar el servidor provisional de inicialización.

.github/workflows/ci.yml prepara Go y Node con las versiones fijadas y ejecuta el mismo `make ci`. Las acciones están fijadas por commit, el permiso general es lectura del repositorio y el trabajo dispone de 25 minutos. GitHub privado y su CI están autorizados; los datos usados son inventados.

La puerta actual no contiene un análisis Semgrep general ni reconstruye toda imagen de producción. Son comprobaciones adicionales exigibles según el cambio, aunque el TRD las enumere entre las puertas de seguridad y despliegue. Ejecuta Semgrep local sobre lo modificado con las reglas versionadas pertinentes y sin subida a su nube. Revisa dependencias y la imagen cuando las cambies.

Demo y vídeos

```
make demo
make demo-comprobar
make demo-recorrido
make demo-parar
```

La demo tiene sus propios contenedores, redes, volúmenes y usuarios inventados. Si el puerto está ocupado al crearla, elige uno libre con `ETF_DEMO_PUERTO=18214 make demo`. El puerto se conserva. La carpeta privada `despliegue/demo/.local/` guarda las cuentas y la configuración; no la añadas a Git.

Parar conserva datos y versiones. `make demo-limpiar` los borra: úsalo sólo cuando quieras reiniciar esa demostración. Al arrancar o actualizar, la preparación para la aplicación, sustituye la base de idiomas, temas y plantillas de la distribución, conserva lo publicado en `.versiones` y lo cambiado a mano y valida los textos (FX8); no exige limpiar los datos. Su ensayo forma parte de la CI: parte de una demo con textos anteriores a SEG1, sin manifiesto y con una versión de idioma activa antigua, y comprueba el Word de Consulta, la emisión Word con una plantilla nueva y las huellas de lo publicado. Los preparadores de recorridos renuevan únicamente sus recursos de distribución cuando el contrato lo permite.

Para vídeos, consulta la guía y los guiones:

```
make video RECORRIDO=11-ciclo-completo
make videos RECORRIDOS="09-memoria 10-carga-2026"
```

Las grabaciones conservan un caso nuevo por ejecución y sus emisiones. Los MP4 se guardan fuera de Git, en la carpeta de vídeos indicada por la herramienta. Revisa duración, fotogramas y capítulos. El reloj de formación se usa sólo mediante el overlay local del recorrido y con datos inventados; no cambia el reloj de identidad ni el código de producción.

La locución de textos redactados por el proyecto tiene autorización específica para el servicio de voz documentado. No se envían datos de expedientes ni contenido introducido por personas usuarias. Los audios se construyen antes y se sirven como archivos estáticos; la app no llama al servicio de voz durante el uso.

Capturas y PDF

Con la demo arrancada:

```
node docs/desarrollo/capturar.mjs
TMPDIR="$PWD/bin/temporales" python3 -B docs/manuales/exportar.py --manual programadores
python3 -B scripts/ci/enlaces.py
```

El capturador exige que la dirección sea la demo local de este worktree. Lee sus cuentas inventadas, abre `/usr/bin/google-chrome` con Playwright y guarda Plantillas Word y Memoria a 1440 y 390 píxeles en `docs/desarrollo/capturas/`. Comprueba sesión, respuestas API, errores JavaScript y ausencia de desbordamiento horizontal. No publica contraseñas ni conserva sesiones de navegador.

Revisa cada imagen y su texto alternativo. Abre el PDF, comprueba diagramas, código, paginación y legibilidad. El exportador conserva por defecto la selección de los tres manuales de usuario; el modo `--manual programadores` sólo escribe este manual. EVIDENCIA.md recoge la validación de esta edición.

Windows y copia pública

```
make paquete-windows
```

El paquete Windows incluye imágenes, demo y lanzadores para Windows 10/11 con WSL2 y Podman Desktop. Genera ZIP versionado y SHA-256 en `bin/paquete-windows/`. Crea accesos que abren Edge o Chrome en ventana propia. Actualizar conserva datos y credenciales y ejecuta la preparación de roles antes de migrar; reiniciar exige confirmación y borra sólo la demo. El estado y límites del ensayo están en WIN1. Este paquete es de pruebas, con datos inventados.

La rama consultada no contiene un generador ni una guía de copia pública. No hay un comando de publicación que pueda documentarse como disponible. El ZIP de pruebas Windows tampoco equivale a una autorización para publicar el repositorio. Si se encarga una copia pública, habrá que fijar su alcance y revisar archivos, licencias, historial, capturas y ausencia de secretos o datos personales antes de prepararla. Este manual no publica contenido ni habilita acceso externo a la aplicación.

Convenciones y entrega de cambios

Los nombres de dominio van en castellano. Mantén paquetes y responsabilidades pequeñas; usa puertos para todo sistema externo. Formatea Go con `gofmt`, utiliza `gopls` para definiciones y referencias y conserva los tipos TypeScript del contrato. Las claves de error identifican la causa sin redactar el mensaje dentro del código. Los comentarios explican decisiones o límites que no se deducen de la implementación.

Actualiza en el mismo cambio los documentos de producto y las guías que describan el comportamiento modificado. Regenera `docs/producto/INDICE.md` con su script después de editar esos documentos. Los títulos de commit van en castellano y describen la capacidad entregada, por ejemplo `Añade consulta histórica de menores al cálculo mensual`. Incluye las atribuciones exigidas por el encargo.

Antes de entregar, integra el `main` local actual en la rama asignada, resuelve conflictos conservando ambas capacidades y repite las comprobaciones afectadas. Termina con `make ci` completo. Deja commits limpios, sin hacer push ni integrar tu rama en `main`. La entrega indica rama, commits, archivos, pruebas reales, documentos actualizados y límites pendientes.

9. Recetas de ampliación

Estas recetas siguen las fronteras de Expedientes ETF. Antes de aplicarlas, localiza una capacidad próxima y sus contratos. Si el cambio necesita una regla funcional aún sin decidir, resuelve esa regla con el responsable antes de guardar datos o publicar una opción.

Añadir un campo a la ficha

1. Define quién lo lee y cambia, si admite desconocido, su validación y la fecha de vigencia. Si afecta a Memoria o SPAF, identifica apartado, corte y tratamiento de ausencia en el contrato estadístico.
2. Añade el valor y sus reglas en `internal/dominio/expediente/`. Conserva la distinción entre desconocido, cero y colección vacía. Evita importar HTTP o PostgreSQL.
3. Amplía el contrato del puerto de entrada y la lectura necesaria del puerto de salida. Define cómo participa el campo en la versión del expediente y en la historia; el servidor conserva la identidad de sesión.
4. Reserva una migración de Expediente. Añade columna o estructura histórica, restricciones y permisos, y actualiza la proyección publicada si el dato lo consume otro módulo. Declara sus dependencias en el manifiesto.
5. Implementa el repositorio y caso de uso con el mismo contexto transaccional. Comprueba ámbito, versión, vigencia, idempotencia cuando corresponda y evento de auditoría sin el valor personal.
6. Amplía el handler y los tipos del cliente de la ficha. Añade etiqueta, ayuda y errores a `idiomas/es/textos/` y `idiomas/en/textos/`; usa `t(clave)` y tokens del tema.
7. Si el campo interviene en documentos o cálculos, conecta productor, puerto, dato de dominio y salida. Prueba un corte anterior y otro posterior a su cambio de vigencia.
8. Ensaya guardado, recuperación, conflicto, equipo ajeno y lectura de Coordinación. Añade las pruebas de SQL y cálculo que correspondan, actualiza documentos y ejecuta `make ci`.

Puntos de partida: familia y adultos, adaptadores PostgreSQL, ficha web y lecturas históricas.

Añadir una pantalla

1. Define ruta, perfiles, estado vacío y acción principal en el flujo de producto. Comprueba que exista el puerto de entrada para consultar o guardar los datos necesarios.
2. Crea la carpeta de capacidad en `web/src/`, su componente, tipos y cliente API. Recibe `PropsPantalla`; conserva borrador, versión y clave de recuperación de las acciones.
3. Añade `pantallas.ts` en esa carpeta con ruta, perfiles, clave de menú y componente. Si cuelga de otra entrada, utiliza `menuDe` según el contrato del registro.
4. Añade la lista a `web/src/pantallas.ts` en el orden acordado. Declara la SPA y sus endpoints en el módulo HTTP, con métodos y permisos completos, y compón la capacidad en `cmd/etf/modulos.go` si es nueva.
5. Publica todas las claves en los directorios de textos de los idiomas mantenidos. Usa variables de tema y controles accesibles. Comprueba que el menú requiera también la pantalla publicada por la sesión.
6. Prueba navegación directa, anónimo, perfil incorrecto, carga, vacío, error, conflicto y salida con cambios sin guardar. Recorre teclado, escritorio, 390 píxeles y alto contraste; contrasta el registro con F0.11.
7. Actualiza flujo, diseño y estado de la capacidad. Ejecuta tipos, pruebas web, textos, navegador y `make ci`.

Referencia pequeña: pantalla de Plantillas, registro de pantallas y composición HTTP.

Añadir una versión de plantilla

1. Elige tipo e idioma y consulta la activa y su revisión en `GET /api/admin/plantillas`. Conserva la versión previa de la referencia para detectar activaciones concurrentes.
2. Prepara un DOCX con versión nueva y el contrato de campos y bloques del tipo. Mantén los textos fijos acordados. Comprueba los marcadores incluso si Word los divide entre fragmentos.
3. Revisa el DOCX con el motor local y sus pruebas: XML, límites, contenido activo y recursos permitidos. Prueba valores con caracteres especiales y colecciones vacías.
4. Desde Administración, sube la versión en Plantillas Word. La API usa `POST /api/admin/plantillas/versiones`, cuerpo DOCX, parámetros de tipo, idioma, versión y versión previa, y cabecera CSRF. Revisa las listas de marcadores faltantes y sobrantes.
5. Confirma que la versión aparece en el historial y que la activa anterior sigue igual. Después activa la elegida con su versión previa actual. Ante 409, recarga y revisa qué activó otra persona.
6. Genera un Word real con datos inventados, comprueba emisión, huella, formato y recuperación idempotente. Comprueba también que una emisión histórica conserva la plantilla anterior.

No sobrescribas el DOCX activo desde el sistema de archivos. Si la versión forma parte de la distribución, registra la actualización mediante una migración reservada de Documento que conserve decisiones administrativas, como hacen las actualizaciones existentes.

Añadir un tipo de documento

1. Define datos, marcadores, bloques, perfiles, idioma y si necesita registro de salida. Separa emisión de confirmación de firma o fecha.
2. Añade el contrato de dominio y puertos de Documento y el lector autorizado de los datos del módulo propietario. Implementa generación con versiones fijadas y escape XML.
3. Reserva la migración para metadatos, tipo y referencias iniciales de plantillas. Incluye RLS y permisos de emisiones y registros si el tipo los necesita.

4. Prepara las plantillas de los idiomas mantenidos y sus huellas. Añade la ruta POST con CSRF, idempotencia y recibo; conecta después el botón traducido de la pantalla.
5. Prueba permiso por tipo, recuperación, reproducción imposible, error de auditoría, concurrencia y ausencia de archivos generados persistentes. Revisa el Word y actualiza los contratos antes de ejecutar la CI.

Fuentes de trabajo: puertos de Documento, motor y migraciones.

Añadir un idioma

1. Crea `idiomas/<codigo>/idioma.json` con código válido, nombre y versión. Añade `textos/`, `catalogos/` y `plantillas/` con la estructura de un idioma mantenido.
2. Traduce todas las claves de pantalla y error conservando parámetros. Comprueba la paridad con `make textos`. No añadas condicionales de idioma al componente para suplir traducciones ausentes.
3. Traduce etiquetas de catálogo conservando los códigos y contratos de versión. Prepara las plantillas del idioma que vaya a ofrecerse para cada tipo disponible; un idioma de pantalla no garantiza un Word traducido.
4. Valida el paquete y publícalo mediante la capacidad de idiomas, con antivirus y versión inmutable. Usa la recarga y activación que expone la administración de paquetes. Confirma que el selector devuelve la referencia activada.
5. Recorre entrada, formularios, fechas, números, errores, documentos y aviso sin conexión. Comprueba `lang`, localización, desbordamientos y respaldo al idioma predeterminado.
6. Conserva las versiones anteriores, actualiza la documentación de idiomas admitidos y ejecuta textos, temas, recorridos afectados y CI.

Revisa el adaptador de paquetes, publicación y la administración web. No registres traducciones en PostgreSQL.

Añadir un catálogo o una opción

1. Comprueba si la opción cabe en un ámbito existente. Fija código estable, vigencia, perfiles de edición y equivalencias que consuman evaluación o estadística.
2. Para una opción de un catálogo existente, utiliza su borrador y publicación administrativa. Para un ámbito o contrato nuevo, amplía dominio, puertos y validaciones; reserva migración sólo cuando cambie estructura o datos de distribución.
3. Publica las etiquetas por idioma mediante los contratos de paquetes. Conserva versiones históricas y verifica que una etiqueta no cambie el significado del código.
4. Conecta el selector a la lectura autorizada del catálogo. El navegador envía el código; el caso de uso valida que sea admisible en la fecha de la operación.
5. Prueba borrador, publicación, vigencia anterior y posterior, desconocido y concurrencia. Si hay equivalencia estadística, comprueba un cálculo pasado y una revisión cerrada.
6. Ejecuta comprobaciones de catálogos, textos, ensayo PostgreSQL cuando corresponda y `make ci`. Actualiza el esquema de datos y la receta funcional de administración afectada.

Puntos de partida: módulo Catálogo, textos de catálogo y comprobador.

Añadir una migración

1. Consulta la reserva externa del proyecto y el mecanismo del encargo. Reserva número antes de crear el fichero; registra módulo, número, rama, descripción, fecha y `Reservada`.

2. Crea `migraciones/<modulo>/NNNN_descripcion.sql`. Comprueba estado previo, integridad y compatibilidad; añade permisos, RLS y `search_path` de funciones cuando corresponda. No incluyas contraseñas.
3. Añade su entrada al manifiesto con `numero`, `archivo` y `depende_de`, aunque la lista sea vacía cuando el contrato lo permita. Declara las dependencias reales por número y evita ciclos.
4. Ejecuta las pruebas del plan y el ensayo del módulo sobre PostgreSQL 18 limpio y actualizado. Comprueba también rechazo de historial alterado, transacciones, aislamiento y datos previos.
5. Obtén revisión independiente de SQL y permisos y ensayo en el clon local con datos inventados. Registra el resultado, no sólo el comando previsto.
6. Integra `main` antes de entregar. Si apareció una migración superior en el mismo módulo, pide número nuevo al director y renombra SQL, manifiesto y pruebas. El migrador rechazará el retroceso.
7. Actualiza el esquema y los documentos de despliegue afectados, regenera el índice y ejecuta `make ci`. Una migración aplicada se corrige con otra posterior; nunca se edita para hacer coincidir una huella.

```
go test ./internal/plataforma/migracion
python3 -B scripts/ci/indice/generar.py
python3 -B scripts/ci/enlaces.py
make ci
```